Design Voyager

Autonomous Game Design via LLM-Powered Mechanic Discovery and Composition

AI For Games Project

Benjamin Tian, Cody Jiang, Morgan Waddington, Ruimeng Yang, Ziyi Zhu

Introduction

Introduction

Problem

1. Knowledge Retention

Current game design processes lack effective knowledge retention, forcing designers to 'reinvent the wheel' and start from scratch with every new project.

2. Creative Limitations

Human creativity is naturally constrained by knowledge boundaries and creative fatigue. When inspiration runs dry, the design process inevitably hits a bottleneck.

Design Voyager:

- **Automated Generation and Evaluation:** Employs LLMs to implement new mechanics, followed by rigorous evaluation using MCTS across dimensions.
- **Mechanics Accumulation:** Archives successfully verified mechanics into a growing library, ensuring valuable design knowledge is saved for future retrieval.
- **Accelerated Design:** Retrieves verified mechanics from the library over iterations to synthesize **good** games, effectively eliminating the "start from scratch" bottleneck.

Introduction

Inspiration

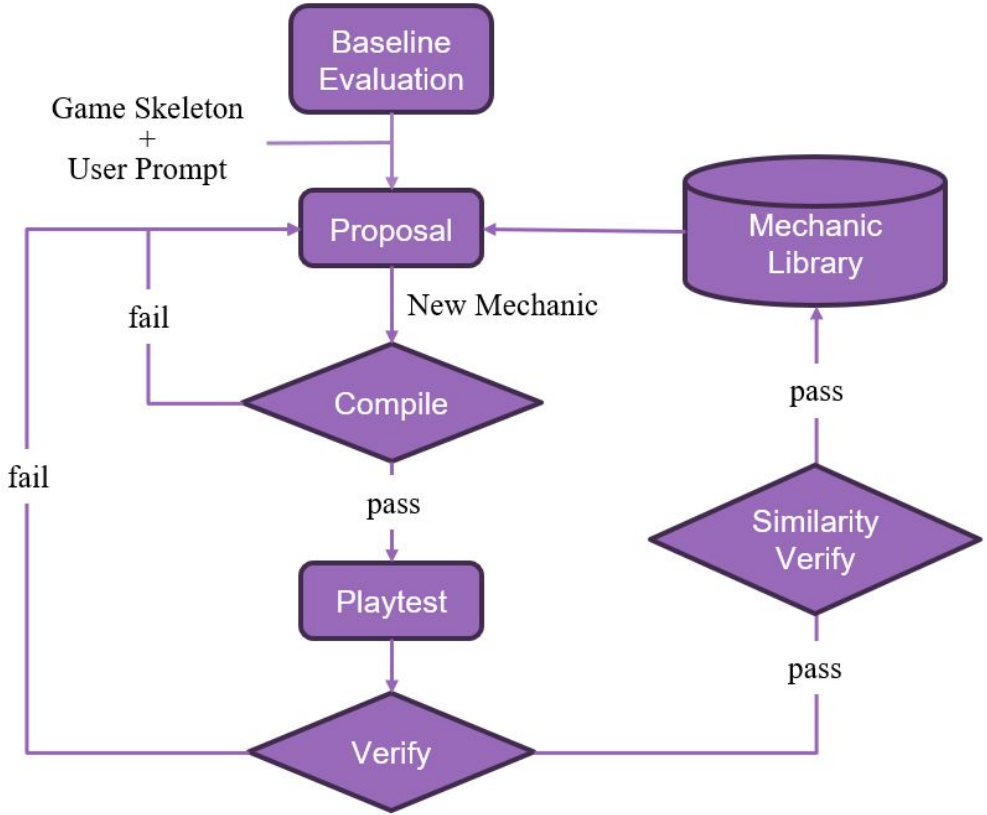
- **Persistent Growing Mechanic Library:** Voyager, a lifelong learning agent for Minecraft playing that continuously discovers and stores skills as executable code, building on prior knowledge to tackle increasingly complex tasks.
- **Adaptive Self-Repair Mechanism:** React, utilizes an action-observation loop where the agent uses environmental feedback (e.g., error logs) to dynamically reason and iteratively correct its own code.

Key Innovation and Customization

- **Diversity-Aware Library Management:** It employs semantic deduplication and filtering to prevent redundancy, ensuring the growth of a high-quality, diverse mechanic library.
- **Lightweight Curriculum:** It guides the agent progressively from simple to complex mechanics. This prevents premature attempts at overly complex designs, thereby avoiding systemic collapse and unplayable outcomes.

Method

Method


Baseline Evaluation
Game Skeleton + User Prompt
Proposal
Mechanic Library
fail
New Mechanic
Compile
pass
fail
pass
Similarity Verify
Playtest
pass
Verify

NYU

Method

Proposal Module

Powered by Gemini 2.5 Flash

- **Prompt:** Specifies the state dictionary and rigorous form constraint, followed by user prompt.
- **Context:** Assembles four context blocks—game skeleton, retrieved mechanics, mechanic library roster, and a ban list.
- **Compile Check:** Two-stage gate invalid syntax in the code

Playtest Module

Evaluates the base game skeleton, followed by playtests the newly integrated mechanics.

- **Search Engine:** Employs MCTS and minimax agents with tactical preambles and a 100-turn cap to simulate play.
- **Two-Phase Test:** Evaluates playability and balance (equal agents), and strategic depth (strong vs. weak agents).

Method

Delta-Gated Verification

Compares playtest metrics against a no-mechanic baseline to determine final acceptance.

- **Simple Gates:** Validates runtime stability and enforces absolute minimum thresholds on key metrics to filter out design regressions.
- **The Delta Gate:** Computes a weighted relative score against a baseline using stage-aware thresholds that tighten as the agent progresses through the curriculum.
- **Outcomes:** Assigns a status of Accept, Discard, or Revise, with generating a natural-language critique to guide the self-repair process.

Adaptive Self-Repair

- **Repair:** Revise mechanics that fail the Compile Check or Delta-Gated Verification in first try, guided by natural language feedback.
- **Discard:** Discard mechanics requiring a second repair attempt and log them in the ban list.

Method

Mechanic Library

- **Storage:** Archives every accepted mechanic as a JSON entry, complete with its description, code, playtest scores, and vector embeddings.
- **Retrieval:** Retrieves *top-k* mechanics using a weighted formula combining semantic similarity (0.7) with prompt and aggregate performance score (0.3), alongside a per-run usage cap to enforce rotation.
- **Diversity Enforcement:** Combats redundancy by exposing a full roster of accepted mechanics to the proposer and enforcing a strict cosine-similarity threshold (<0.92) before final archiving.

Lightweight Curriculum

- **Progressive Stages:** Defines three complexity tiers, which let system propose mechanic from easy to complex; advancement is triggered by three consecutive accepted mechanics.
- **Stage-Aware Validation:** Applies dynamic verification thresholds that tighten as the agent progresses through curriculum stages.

Method

Pair-Lab Evaluation

Assesses the system's ability to generate good games, rather than just the quality of individual mechanics.

- **Expanded Metric:** Evaluates each pair against an expanded vector of eight quality metrics, derived from per-game telemetry.
- **Interaction Focus:** The new metrics measure mechanic interaction depth to provide a comprehensive assessment of the overall game quality.

Metric	Definition
balance	$1 - \lvert p_1$ win rate $- p_2$ win rate$\rvert$ over decisive games
decisiveness	Fraction of games that ended with a winner reaching the target score
both_meaningful	Minimum per-mechanic fire-rate-by-game across the pair
length_sanity	1.0 if average game length lies in $[8, 30]$ turns; scaled outside
volatility	Average lead changes per game, capped at six and normalised
length_cv	Coefficient of variation of game length (stdev divided by mean)
joint_fire_rate	Fraction of turns on which every mechanic in the pair fired
non_greedy_rate	Fraction of moves where the agent did not play its highest-value card

Fitness Function

- **Necessity:** Bridges the gap between objective data and subjective player experience
- **Impact:** Aligns metric-based rankings with actual human preferences. This proves the system can design games that humans enjoy, while providing a reliable ranking reference for the generated games.

Results

Thank you!

Design Voyager: Autonomous Game Design via LLM-Powered Mechanic Discovery and Composition